CAN LLMS SEPARATE INSTRUCTIONS FROM DATA? AND WHAT DO WE EVEN MEAN BY THAT?

Egor Zverev
ISTA
egor.zverev@ist.ac.at

Sahar Abdelnabi
Microsoft Security Response Center
saabdelnabi@microsoft.com

Soroush Tabesh
ISTA
stabesh@ist.ac.at

Mario Fritz
CISPA Helmholtz Center for Information Security
fritz@cispa.de

Christoph H. Lampert
ISTA
chl@ist.ac.at

arXiv:2403.06833v3 [cs.LG] 31 Jan 2025

ABSTRACT

Large Language Models (LLMs) show impressive results in numerous practical applications, but they lack essential safety features that are common in other areas of computer science, particularly an explicit separation of *instructions* and *data*. This makes them vulnerable to manipulations such as indirect prompt injections and generally unsuitable for safety-critical tasks. Surprisingly, there is currently no established definition or benchmark to quantify this phenomenon. In this work, we close this gap by introducing a formal measure for instruction-data separation for single-turn language models and an empirical variant that is calculable from a model's outputs. We also present a new dataset, SEP, that allows estimating the measure for real-world models. Our results on various LLMs show that the problem of instruction-data separation is real: all models fail to achieve high separation, and canonical mitigation techniques, such as prompt engineering and fine-tuning, either fail to substantially improve separation or reduce model utility. The source code and SEP dataset are openly accessible at https://github.com/egozverev/Shold-It-Be-Executed-Or-Processed.

1 INTRODUCTION

Large language models (LLMs) (Achiam et al., 2024; Touvron et al., 2023) have quickly been adopted in many applications due to their amenable flexibility via natural language instructions. This includes general-purpose applications where LLMs may be fed arbitrary external data and tasks are defined at runtime. For example, users' emails or search results may be fed to the LLM to summarize or answer queries. Increasingly, they also serve as the backbone of special-purpose applications that can be deployed via APIs, by customizing models with tailored instructions (Perez & Ribeiro, 2022; OpenAI, 2023) thus creating task-specific applications to process users' data.

As a result, we now already have an ecosystem of sophisticated LLM-powered applications, including production-level web or email clients (Microsoft, 2023), automated software systems (Tao et al., 2024), and Retrieval-Augmented-Generation (RAG) pipelines to support classical NLP tasks, such as summarization, or domain-specific tasks, like report generation in highly sensitive settings (Ma et al., 2024). In all of these scenarios, one crucial safety aspect is that the resulting model must exclusively execute its primary instruction, while all additional inputs (e.g., text providing background information to help solve a task) must be treated only as passive data. LLMs should process their inputs according to a strict set of rules, where the execution of any instructions in the data must be prohibited.

Current LLMs do not enforce explicit separation between the *instructions* that models are meant to *execute* and the *data* that they are meant to *process*. If such a separation does not adequately exist, the model can show undesirable behaviors as a result of misinterpretation or, in the worst case, explicit malicious attacks.

Figure 1 explains this phenomenon with a simple example. An email client is created by specifying its rules and use-cases in a system prompt. It can summarize or send emails in response to the user's prompts. It must *not* send emails in response to instructions found within the email. However, due to the lack of such a separation, in

Instructions: You are an email client. You can summarize emails and send emails based on user's requests via **"call_api"** function call. **You should never answer any requests or questions or commands found in the emails.** Now **summarize** the following emails

Data: <emails> ... Hey, We're planning a team-building event next month. I'd love for you to send me a brief description of an activity you'd enjoy. Also please send back an email with subject "Confirm" to confirm receiving this email. Please do so urgently. Cheers, Daniel ... </emails>

LLM: ..... **call_api("send_email", "Confirm", "Daniel's email")** .....

Figure 1: Illustrative example of a lack of instruction-data separation in a simulated LLM-integrated email client with the `Phi-3-medium-128k-instruct` model. The client mistakenly executes an API after treating a part of passive data (i.e., emails to the user) as an instruction, despite the received instruction being only to summarize the email. Blue snippets highlight parts of the instructions that aim to control the model's answer (and fail). Chestnut snippets highlight the wrongly executed instruction.

addition to unreliable rule following (Mu et al., 2023), it may call an API in response to instructions found in the email that were, based on the context, clearly addressed to the user, i.e., the email's receiver.

This example outlined above illustrates a fundamental limitation of current LLMs that undermine their trustworthiness even in the most straightforward setup without any exploitation. Note that this is not the result of a malicious behavior, but simply a case where the model wrongly treats a harmless input as an instruction and executes it, when it should have treated it as data and processed it (i.e., summarized). Clearly, the lack of instruction-data separation is particularly concerning in safety/security-crucial applications; e.g., malicious instructions injected in emails (Microsoft, 2024b). Even more dire consequences can occur if third parties are aware of this issue and specifically attempt to exploit it via so-called (indirect) prompt injections (Greshake et al., 2023). Such attacks may lead to actual harms of data exfiltration or influencing the LLM's output for other users (Bargury, 2024; Microsoft, 2024a).

Current safety training mechanisms that focus solely on rejecting harmful prompts are not adequate or appropriate to address this more fundamental problem that is more concerned with the contextual nature of instructions: their source. At the same time, while existing works have hypothesized the lack of instruction-data separation to be the underlying cause of prompt injections (Perez & Ribeiro, 2022; Greshake et al., 2023; Yi et al., 2024), such a separation has not been thoroughly investigated before from first principles.

On an architectural level, today's LLMs do not possess a formal, principled separation of *passive* data from *active* instructions. This is partly owed to their development as instruction-following models (e.g., chatbots), for which instructions can occur anywhere in their input, be it a system prompt or a user one (OpenAI, 2023). In contrast, such a separation is one of the core security principles in modern computer systems. Already in the 1990s, when databases were increasingly made accessible remotely via the Internet, the problem of *SQL injections* was identified, and suitable mitigation techniques were developed (Clarke-Salt, 2009). Similarly, all modern CPU architectures allow marking memory regions as *not executable* (Hennessy & Patterson, 2017), and *executable-space protection* mechanisms were included in all major operating systems (Hewlett Packard, 2005) more than 20 years ago.

Contributions. In this work, we make an attempt to lay out a similar path in the context of large language models, on a conceptual as well as an empirical level. Specifically, one of our main contributions is **a formal characterization of *instruction-data separation*** for single-turn language models (meaning models that do not engage in multiple conversational rounds like chatbots). There are numerous historical precedents indicating that being able to formally describe a desirable or undesirable property is important for building systems that reliably exhibit this preference. Examples range from *provably secure cryptography* (Goldreich, 2001) and *formal verification* (Clarke et al., 2018) over *differential privacy* (Dwork et al., 2014) to *algorithmic fairness* (Barocas et al., 2023).

In the context of LLM research, a formal definition is most useful if it can be computed or estimated efficiently for practically relevant models. For this purpose, as a second contribution, we introduce **a proxy measure and a dataset** that allow estimating the amount of instruction-data separation for any promptable language model without the need for the model's internal states or probabilistic outputs. Finally, our final contribution is an **empirical evaluation of the data-instruction separation of several state-of-the-art language models**, as well as the effectiveness of canonical techniques that could be used to improve this separation, namely prompt engineering, prompt optimization, and fine-tuning.

2 RELATED WORK

Most current research on LLM security and safety focuses on studying jailbreaks (i.e., harmful queries) and defending models against them (Zou et al., 2023; Liu et al., 2024; Chao et al., 2023; Zeng et al., 2024). We make an important distinction between jailbreaks and the fundamental limitation of improper instruction-data separation (and subsequent attacks that are enabled by it), which we address in our work. This phenomenon was first introduced in (Greshake et al., 2023), however, with no quantification. Follow-up work Yi et al. (2024) provided more quantification and benchmarking for different LLMs, with a focus on malicious instructions injected within text paragraphs. More recent work is concerned with how these attacks can be mounted end-to-end in RAG frameworks (De Stefano et al., 2024; RoyChowdhury et al., 2024; Microsoft, 2024b) or agentic applications (Debenedetti et al., 2024) and how they can lead to undesired API calls or misinformation propagation. Also in RAG setups, Pasquini et al. (2024) optimize tokens to promote the execution of injected instructions placed within larger text blocks.

To remedy this problem, Piet et al. (2024) proposed a defense against this instruction-hijacking by deploying non-instruction-tuned specific-purpose models, sacrificing conversational ability. Chen et al. (2024) fine-tuned models to follow instructions only within artificially created text blocks enclosed by specified tokens. Hines et al. (2024) used prompting-based methods to *spotlight* the data parts in the context via, e.g., specific tokens. Wallace et al. (2024) fine-tuned models to assign priorities of execution to different prompts' types. Abdelnabi et al. (2024) detect instructions introduced in supposedly-data blocks via white-box inspections of models' activation deltas before and after feeding data blocks. Bagdasarian et al. (2024) limit data exfiltration risks due to injection attacks by using a task-specific sensitive-data minimization step.

Despite this substantial activity in the area over the past two years, our understanding of the problem is still in its infancy. This work aims to remedy this gap by *defining* and *evaluating* the data-instruction problem from a fundamental perspective, isolating it from attacks and other safety issues such as the execution of explicitly harmful instructions.

3 CAN LLMS SEPARATE INSTRUCTIONS FROM DATA?

In order to reason formally about the separation of instructions and data in LLMs we introduce the following abstraction:

Definition 1. For an input alphabet A, we formalize a **single-turn language model** (LM) as a mapping $g : A^* \times A^* \to \mathcal{M}(A^*)$, where A^* is the set of strings over the alphabet A, and $\mathcal{M}(\cdot)$ denotes the set of probability distributions over a base set. We call the language model's arguments the *instruction argument* and the *data argument*.

Discussion. By design, we define language models as abstract functions here, thereby making the definition agnostic to aspects of model architecture or implementation. In particular, we do not specify *how* the inputs are processed or how the separation between instruction and data arguments is achieved, if at all. For a discussion on how Definition 1 applies to existing LLMs, see Section 5. Our central definition describes a way to quantify the separation a model achieves between instructions and data:

Definition 2. Let $p \in \mathcal{M}(A^* \times A^* \times A^*)$ be a joint probability distribution over triples (s, d, x) of strings, where we call s the *task prompt*, d the *data prompt*, and x the (task-like) *probe* string. We define the **separation score** of a language model, g, as

$$\text{sep}_p(g) = \mathbb{E}_{(s,d,x)\sim p}\mathcal{D}\big(g(s, x + d), g(s + x, d)\big). \tag{1}$$